EEM 364
Mikroişlemcileri

Koşullar ve Döngüler ve Prosedürler

Sıralama algoritması

```
.MODEL SMALL

.DATA

x    db 1,3,5,2,2,1,8,0,4, 7
lx equ $-x-1

.CODE

main proc far
     mov ax , @data
     mov ds , ax

L0:
     mov ax , 0
     mov cx , lx
     mov si , 0
     mov dl , 0



L:
     mov al , x[si]
     mov ah , x[si+1]

     call SWP
     inc si
     loop L

     cmp dl , 0

     jnz L0

HLT

main endp

SWP proc

     cmp al , ah
     jle not_swp
     mov x[si+1] , al
     mov x[si]   , ah

     inc dl

     not_swp:
     ret
SWP endp
```

Koşullar ve Döngüler ve

Çarpma ve Bölme

```
.MODEL SMALL

.DATA

A equ  15
C equ  4

Z dw 0


.CODE

main proc far

    mov ax , @data
    mov ds , ax


    mov ax , 0
    mov bx , 0

    call carp

    mov Z , bx

    HLT

main endp


carp proc

    mov al , A
    mov cx , C


    CAR: add bx , ax
         loop CAR


    ret

carp endp
```

```
.MODEL SMALL

.DATA

X equ  53
B equ  7


Z dw 0


.CODE

main proc far

    mov ax , @data
    mov ds , ax

    mov ax , 0

    call bol

    hlt

main endp


bol proc

    mov al , X
    mov bx , 0

    L:   inc ah
         sub al , B
         cmp al , B
         jge L
    ret

bol endp
```

Koşullar ve Döngüler ve Prosedürler !!

- **JNZ/JZ**
- **CMP**
- **INC**
- **DEC**
- **OFFSET**
- **EQU**
- **LEA**

- **CMP**
- **JE , JG , JL**
- **TEST**
- **PROC**
- **LOOP**
- **MUL**

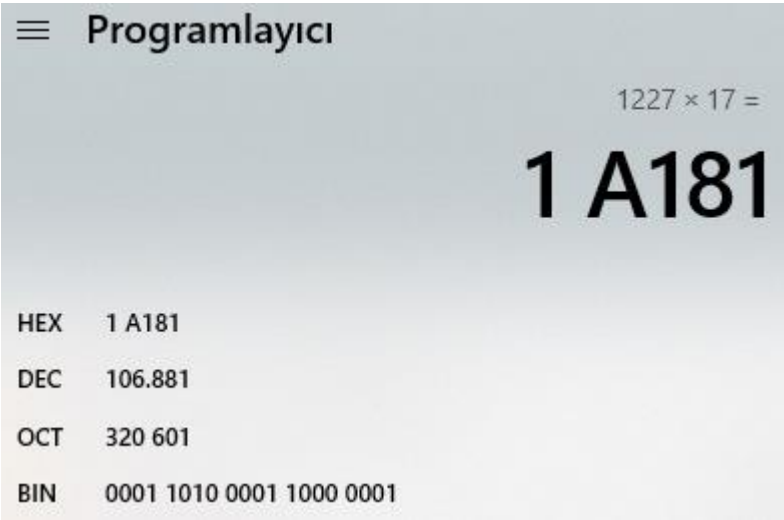
- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

Prosedürler Devam

- **MUL ve IMUL**
- reg , reg
- reg , mem

```
MOV AX , 1227H
MOV DX , 17H
MUL DX
```



- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

YAZMAÇLAR	
AH: 12	AL : 27
AX :A1 81	
DX: 00 01	

Prosedürler Devam

- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

```
MOV AX , 1227H
MOV DX , 17H
MUL DX
```

```
MOV AX , 1227H
MOV DL , 17H
MUL DL
```

```
01 mov ax , 1203H
02
03 mov dl , 2
04
05 mul dl
06
07
```

file math debug

Load

registers

	H	L
AX	00	06
BX	00	00
CX	00	00

Programlayıcı

1227 × 17 =

1 A181

HEX	1 A181
DEC	106.881
OCT	320 601
BIN	0001 1010 0001 1000 0001

YAZMAÇLAR	
AH: 12	AL : 27
AX :A1 81	
DX: 00 01	

Prosedürler Devam

- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan:**
  - AL veya AX
- **Çarpan:**
  - BL,CL,DL
  - BX,CX,DX

```
01 T dw 0
02
03 mov al , 180
04
05 mov dl , 2
06
07 mul dl
08
09 mov T , ax
10
```

size: word elements: 1
edit show as: unsigned

T 360

registers

	H	L
AX	01	68
BX	00	00
CX	00	00
DX	00	02
CS	0100	
IP	000B	

Prosedürler Devam

- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

```
.MODEL SMALL

.DATA

X equ  6

S dw 0


.CODE

mov ax , @data
mov ds , ax


mov ax , 0001H
mov cx , X

L:
mul cx
loop L

mov S , ax


HLT
```

Faktöriyel !

Prosedürler Devam

- **DIV ve IDIV**
- reg , reg
- reg , mem

```
MOV AL , 27H ; 39
MOV DL , 17H ; 23
DIV DL
```

YAZMAÇLAR	
AH: ~	AL : 27
AX: 10 01	

- **Bölünen**:
  - AL veya AX
- **Bölen**:
  - BL, CL, DL
  - BX, CX, DX

Prosedürler Devam

- **DIV ve IDIV**
- reg , reg
- reg , mem

```
MOV AL , 27H ; 39
MOV DL , 17H ; 23
DIV DL
```

YAZMAÇLAR	
AH: ~	AL : 27
AX: 10 01	

- **Bölünen**:
  - AL veya AX
- **Bölen**:
  - BL, CL, DL
  - BX, CX, DX

```
MOV AL , 42H; 66
MOV DL , 12H; 18
DIV DL
```

YAZMAÇLAR	
AH: ~	AL : 42
AX: 0C 03	

Prosedürler Devam

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA

- CMP
- JE , JG , JL , JNS
- TEST
- PROC
- LOOP
- MUL
- DIV

Asal Sayı Bulma

```
.MODEL SMALL

.DATA
x equ 53
y equ x/2

BN db  0
BM db 0
FL db 1

.CODE

main proc far

mov ax , @Data
mov ds , ax

mov cl , 2
mov dx , x

call asal


hlt
main endp

asal proc

    L:
    mov ax , dx
    div cl
    cmp ah , 0
    jz not_asal
    inc cl
    cmp cl , y
    jnz L


    D:

    ret

    not_asal:

    mov BN , cl
    mov BM , al
    mov FL , 0

    jmp D

asal endp
```

İç İçe Döngüler

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA

- CMP
- JE , JG , JL , JNS
- TEST
- PROC
- LOOP
- MUL
- DIV
- PUSH
- POP

İç İçe Döngüler

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA

- CMP
- JE , JG , JL , JNS
- TEST
- PROC
- LOOP
- MUL
- DIV
- PUSH
- POP

Matris Satır Toplama

```
.MODEL SMALL

.DATA


 x   db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx  equ 3
 ly  equ 4

 T db lx dup(0)

.CODE

Main PROC Far

    mov ax , @data
    mov ds , ax



    mov ax , 0
    mov cx , lx

    mov si , 0
    mov di , 0

    L1: push cx
        mov cx , ly
        mov dl , 0

        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

        mov T[di] , dl
        inc di

        pop cx

    loop L1


HLT

Main ENDP
```

```
.MODEL SMALL
.DATA

 x   db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx  equ 3
 ly  equ 4

 T  db lx dup(0)
```

İç İçe Döngüler

İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far

    mov ax , @data
    mov ds , ax
```

```
HLT
Main ENDP
```

İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x   db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx  equ 3
 ly  equ 4

 T db lx dup(0)
.CODE
Main PROC Far

    mov ax , @data
    mov ds , ax

        mov al , x[si]
        add dl , al

HLT
Main ENDP
```

İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far

    mov ax , @data
    mov ds , ax
```

```
        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

HLT
Main ENDP
```

İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far

    mov ax , @data
    mov ds , ax

        mov cx , ly
        mov dl , 0

        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

        mov T[di] , dl
        inc di

HLT
Main ENDP
```

İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far

    mov ax , @data
    mov ds , ax


    mov ax , 0
    mov cx , lx

    mov si , 0
    mov di , 0

    L1: push cx
        mov cx , ly
        mov dl , 0

        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

        mov T[di] , dl
        inc di

        pop cx

    loop L1

HLT
Main ENDP
```

İç İçe Döngüler

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA

- CMP
- JE , JG , JL , JNS
- TEST
- PROC
- LOOP
- MUL
- DIV
- PUSH
- POP

Matris içindeki bir elemanın indisini bulma

```
.MODEL SMALL

.DATA

 x  db 1,2,3,4,  5,6,7,8,  9,10,11,12
 lx equ $-x

.CODE


Main PROC FAR

      mov ax , @data
      mov ds , ax


      mov dh , 0

      L1:


         mov dl , 0

         L2:
             mov al , x[si]
             cmp x[si] , 7
             je K

             inc si


             inc dl
             cmp dl , 4
             jne L2


       inc dh
       cmp dh , 3
       jne L1


      K:
      hlt


Main ENDP
```

Kesmeler

DOS INT 21h - DOS Function Codes

The follow abridged list of DOS interrupts has been extracted from a large list compile

AH	Description	AH	Description
01	Read character from STDIN	02	Write character to STDOUT
05	Write character to printer	06	Console Input/Output
07	Direct char read (STDIN), no echo	08	Char read from STDIN, no echo
09	Write string to STDOUT	0A	Buffered input
0B	Get STDIN status	0C	Flush buffer for STDIN
0D	Disk reset	0E	Select default drive
19	Get current default drive	25	Set interrupt vector
2A	Get system date	2B	Set system date
2C	Get system time	2D	Set system time
2E	Set verify flag	30	Get DOS version
35	Get Interrupt vector		
36	Get free disk space	39	Create subdirectory
3A	Remove subdirectory	3B	Set working directory
3C	Create file	3D	Open file
3E	Close file	3F	Read file
40	Write file	41	Delete file
42	Seek file	43	Get/Set file attributes
47	Get current directory	4C	Exit program
4D	Get return code	54	Get verify flag
56	Rename file	57	Get/Set file date

Yazılım kesmeleri INT makine komutuyla oluşturulan kesmelerdir. Programcı kesme oluşturmak istediği zaman INT makine komutunu kullanabilir. INT makine komutu tıpkı CALL makine komutu gibi program akışını kesme koduna geçirmektedir. INT makine komutuyla oluşturulan kesmeden IRET makine komutuyla geri dönülür.

Donanım kesmeleri mikroişlemcinin dışındaki bir birim tarafından mikroişlemcinin INT ucunun aktive edilmesiyle oluşturulur. Kesme denilince ilk akla gelen donanım kesmeleridir. Mikroişlemci bir makine komutunu çalıştırdıktan sonra, bir sonraki makine

http://www.mentis.com.tr/web/80x86-assembly-version1.pdf

Kesmeler

DOS INT 21h - DOS Function Codes

The follow abridged list of DOS interrupts has been extracted from a large list compile

AH	Description	AH	Description
01	Read character from STDIN	02	Write character to STDOUT
05	Write character to printer	06	Console Input/Output
07	Direct char read (STDIN), no echo	08	Char read from STDIN, no echo
09	Write string to STDOUT	0A	Buffered input
0B	Get STDIN status	0C	Flush buffer for STDIN
0D	Disk reset	0E	Select default drive
19	Get current default drive	25	Set interrupt vector
2A	Get system date	2B	Set system date
2C	Get system time	2D	Set system time
2E	Set verify flag	30	Get DOS version
35	Get Interrupt vector		
36	Get free disk space	39	Create subdirectory
3A	Remove subdirectory	3B	Set working directory
3C	Create file	3D	Open file
3E	Close file	3F	Read file
40	Write file	41	Delete file
42	Seek file	43	Get/Set file attributes
47	Get current directory	4C	Exit program
4D	Get return code	54	Get verify flag
56	Rename file	57	Get/Set file date

```
.model small
.data
    msg db "Hello, World! $"
.code
main proc far
    mov ax , @data
    mov ds ,ax

    mov  dx, offset msg
    mov  ah, 9
    int  21H

    mov  ah, 4CH
    int  21H

main endp
```

Kesmeler

DOS INT 21h - DOS Function Codes

The follow abridged list of DOS interrupts has been extracted from a large list compile

AH	Description	AH	Description
01	Read character from STDIN	02	Write character to STDOUT
05	Write character to printer	06	Console Input/Output
07	Direct char read (STDIN), no echo	08	Char read from STDIN, no echo
09	Write string to STDOUT	0A	Buffered input
0B	Get STDIN status	0C	Flush buffer for STDIN
0D	Disk reset	0E	Select default drive
19	Get current default drive	25	Set interrupt vector
2A	Get system date	2B	Set system date
2C	Get system time	2D	Set system time
2E	Set verify flag	30	Get DOS version
35	Get Interrupt vector		
36	Get free disk space	39	Create subdirectory
3A	Remove subdirectory	3B	Set working directory
3C	Create file	3D	Open file
3E	Close file	3F	Read file
40	Write file	41	Delete file
42	Seek file	43	Get/Set file attributes
47	Get current directory	4C	Exit program
4D	Get return code	54	Get verify flag
56	Rename file	57	Get/Set file date

```
.model small

.data

    msg db "Bir sayi giriniz:  $"

.code

main proc far
    mov ax , @data
    mov ds ,ax

    mov dx, offset msg

    mov  ah, 9
    int  21H


    mov ah , 01H
    int  21H


    mov  ah, 4CH   ; HLT YERINE
    int  21H
main endp
```

LAB ÖDEVİ

- **Bir dizindeki max elemanı bulma**
- **Palindrom: 12321 veya KABAK**
- **Bir dizinin ortalamasını bulmak**

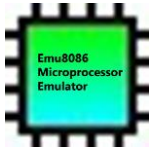
Hafta 13
İç İçe Döngüler Kesmeler ve Makrolar

```
; dizi icerisinde en yuksek degeri bulmak
.model small

.data

 x    db 12,23,2,4,8,1,2,32,14,76,99,14,53,32,44
 lx  equ $-x
 MX   db 0

.code
 mov ax , @data
 mov ds , ax

 mov ax , 0
 mov si , 0

 L:
 mov al , x[si]
 cmp MX , al
 jle degis
 D:
 inc si
 cmp si , lx
 jnz L


HLT

degis:
mov MX , al
jmp D
```

```
; palindrom tespiti
.model small

.data

 x    db 1,2,3,4,0,3,2,1
 lx  equ ($-x)
 lh  equ lx/2

 F    db 1   ; sizi ilk olarak palindrom
             ; olarak tanimlanir

.code
 mov ax , @data
 mov ds , ax

 mov ax , 0
 mov si , 0
 mov di , lx-1

 L:
 mov al , x[si]
 mov ah , x[di]

 cmp al , ah
 jnz D

 dec di
 inc si
 cmp si , lh
 jnz L


SON:


HLT

D:
mov F , 0
jmp SON
```

İç İçe Döngüler

Diagonal Toplam



İç İçe Döngüler

Matris içinde belli bir elemanı sayma

İç İçe Döngüler

Alt Üçgen Toplam

Kesmeler

DOS INT 21h - DOS Function Codes

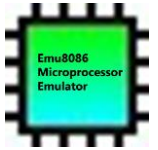
The follow abridged list of DOS interrupts has been extracted from a large list compile

AH	Description	AH	Description
01	Read character from STDIN	02	Write character to STDOUT
05	Write character to printer	06	Console Input/Output
07	Direct char read (STDIN), no echo	08	Char read from STDIN, no echo
09	Write string to STDOUT	0A	Buffered input
0B	Get STDIN status	0C	Flush buffer for STDIN
0D	Disk reset	0E	Select default drive
19	Get current default drive	25	Set interrupt vector
2A	Get system date	2B	Set system date
2C	Get system time	2D	Set system time
2E	Set verify flag	30	Get DOS version
35	Get Interrupt vector		
36	Get free disk space	39	Create subdirectory
3A	Remove subdirectory	3B	Set working directory
3C	Create file	3D	Open file
3E	Close file	3F	Read file
40	Write file	41	Delete file
42	Seek file	43	Get/Set file attributes
47	Get current directory	4C	Exit program
4D	Get return code	54	Get verify flag
56	Rename file	57	Get/Set file date

```
.model small

.data

    msg db "Bir sayi giriniz:  $"

.code

main proc far
    mov ax , @data
    mov ds ,ax

    mov dx, offset msg

    mov  ah, 9
    int  21H


    mov ah , 01H
    int  21H


    mov  ah, 4CH  ; HLT YERINE
    int  21H
main endp
```

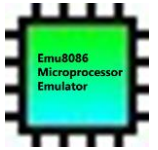
Kesmeler

5 sayı girme



Kesmeler

5 sayı toplama



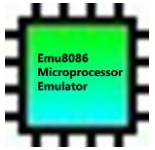
Kesmeler

Yeni Satır

```
01 .model small
02
03 .data
04
05     msg1 db "5 sayi giriniz:   $"
06     msg2 db "2 sayi giriniz:   $"
07
08     ys db 0AH , 0DH , '$'
09
10 .code
11
12 main proc far
13     mov ax , @data
14     mov ds ,ax
15
16     lea  dx, msg1
17     mov  ah, 9
18     int  21H
19
20
21     lea  dx, ys
22     mov  ah, 9
23     int  21H
24
25
26     lea  dx, msg2
27     mov  ah, 9
28     int  21H
29
30     hlt
31
32 main endp
```

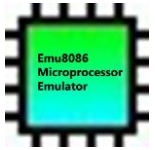
Kesmeler

«s» Karakter kontrolü



Kesmeler

«s» gelinceye kadar
«a» sayma



Makrolar

emu8086.inc defines the following **macros**:

- **PRINT string** - macro with 1 parameter, prints out a string.
- **PRINTN string** - macro with 1 parameter, prints out a string. The same as PRINT but automatically adds "carriage return" at the end of the string.
- **PUTC char** - macro with 1 parameter, prints out an ASCII char at current cursor position.
- **GOTOXY col, row** - macro with 2 parameters, sets cursor position.
- **CURSOROFF** - turns off the text cursor.
- **CURSORON** - turns on the text cursor.

Makrolar

```
include 'emu8086.inc'
.model small
.data
.code
main proc far


    mov cl , 10
    mov ch , 0

    mov bl , 0
    mov bh , 8


    mov al , 2
    mov ah , 8

    ;gotoxy ch , cl
    ;gotoxy bh , bl
    ;gotoxy bl , al

    putc '*'


    hlt


main endp
```

Makrolar

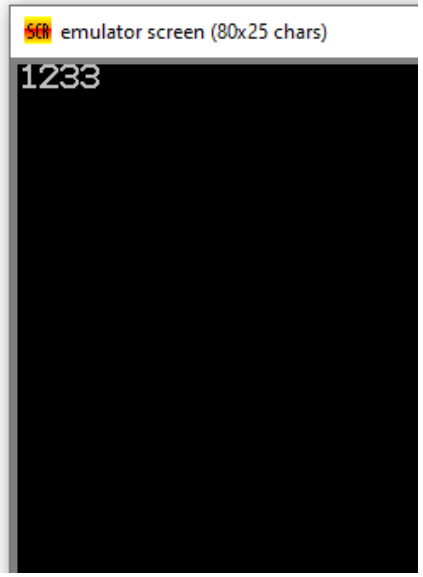
```
include 'emu8086.inc'
.model small
.data
.code
main proc far


    mov cl , 10
    mov ch , 0

    mov bl , 0
    mov bh , 8


    mov al , 2
    mov ah , 8

    ;gotoxy ch , cl
    ;gotoxy bh , bl
    ;gotoxy bl , al

    putc '*'

    hlt

main endp
```

```
; sets current cursor
; position:
GOTOXY  MACRO   col, row
        PUSH    AX
        PUSH    BX
        PUSH    DX
        MOV     AH, 02h
        MOV     DH, row
        MOV     DL, col
        MOV     BH, 0
        INT     10h
        POP     DX
        POP     BX
        POP     AX
ENDM
```

Makrolar

```
include 'emu8086.inc'
.model small
.data
.code
main proc far


    mov cl , 10
    mov ch , 0

    mov bl , 0
    mov bh , 8

    mov al , 2
    mov ah , 8

    ;gotoxy ch , cl
    ;gotoxy bh , bl
    ;gotoxy bl , al

    putc '*'

    hlt

main endp
```

```
.model small
.data
.code
main proc far

mov ax , 1233
call print_num_uns


main endp

DEFINE_PRINT_NUM_UNS
```



Makrolar

emu8086.inc defines the following **macros**:

- **PRINT string** - macro with 1 parameter, prints out a string.
- **PRINTN string** - macro with 1 parameter, prints out a string. The same as PRINT but automatically adds "carriage return" at the end of the string.
- **PUTC char** - macro with 1 parameter, prints out an ASCII char at current cursor position.
- **GOTOXY col, row** - macro with 2 parameters, sets cursor position.
- **CURSOROFF** - turns off the text cursor.
- **CURSORON** - turns on the text cursor.

- **CLEAR_SCREEN** - procedure to clear the screen, (done by scrolling entire screen window), and set cursor position to top of it. To use it declare: **DEFINE_CLEAR_SCREEN** before **END** directive.
- **PTHIS** - procedure to print a null terminated string at current cursor position (just as PRINT_STRING). The ZERO TERMINATED string should be defined just after the CALL. For example:

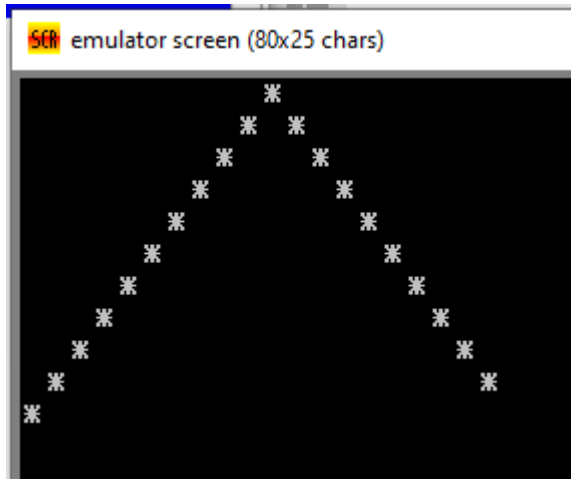
```
CALL PTHIS
db 'Hello World!', 0
```

  Address of string is stored in the Stack as return address. Procedure updates value in the Stack to make return after string definition. To use it declare: **DEFINE_PTHIS** before **END** directive.

- **SCAN_NUM** - procedure that gets the multi-digit SIGNED number from the keyboard, and stores the result in **CX** register. To use it declare: **DEFINE_SCAN_NUM** before **END** directive.

Makrolar

Şekil oluşturma




```
gotoxy ch , cl
putc '*'
inc ch
dec cl
```

Makrolar

Şekil oluşturma

Makrolar

Ekrana sayı basma

```
include 'emu8086.inc'

.model small

.data

.code

main proc far
    mov ax , @data
    mov ds , ax

    mov ax , 15294
    call print_num_uns

main endp

define_print_num_uns
```

LAB Ödevi

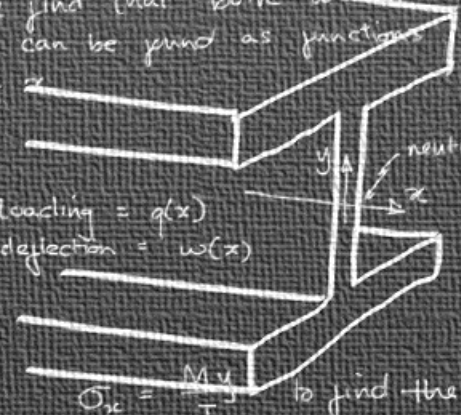
Dışarıdan 5 sayı al $T = \sum_{i=0}^{4} x[i]$ $y[i] = T - x[i]$ $\forall i$

Ters Dizi 1,2,3,4,5 Çıktı 5,4,3,2,1

Histogram a,c,d,b,b,a,a,b,a,c,b,a,b,d,a,b Sayıları ekrana yaz

Dizi arama (a,b) a,c,d,b,b,a,a,b,a,c,b,a,b,d,a,b

given a beam of constant EI
we find that both w and
q can be found as functions
of x



loading = $q(x)$
deflection = $w(x)$

$\sigma_x = \frac{My}{I}$ to find the tension force at some location y

for a static system

$$\frac{\partial^2}{\partial x^2}\left(EI\frac{\partial^2 w}{\partial x^2}\right) = q$$

for $\frac{\partial EI(x)}{\partial x} = 0$

where $\frac{\partial^4 w}{\partial x^4} EI = q(x)$

E = Young's Modulus

I = Second moment of area

http://bsnscb.com/data/out/124/39800806-maths-wallpapers.jpg